

Step 9: Deployment Plan

0. Introduction

In this final phase of the capstone, we outline a deployment and operations plan for our cross-modal image-to-text retrieval service. We begin by describing the offline ETL pipeline that ingests raw images and captions from S3, encodes them using our CLIP-based model in a daily batch job, and stores the resulting embeddings and FAISS index artifacts back to S3. Next, we detail our CI/CD workflow, how code commits trigger GitHub Actions to build and publish Docker images to Amazon ECR, and how ECS Fargate automatically deploys and scales our FastAPI inference containers behind an Application Load Balancer. Finally, we cover the post-deployment guardrails: Service Level Objectives (SLOs) for latency and availability, real-time monitoring and alerting via Prometheus and CloudWatch, automated retraining cadences, canary releases, rollback procedures, and a cost dashboard.

1. Executive Summary

We will expose the “text-to-image-to-caption” retrieval model (ViT-B-32 and FAISS IVF-Flat) behind a Python FastAPI microservice packaged in a Docker container and orchestrated with AWS Elastic Container Service (ECS Fargate).

This proposed service returns the k nearest results (can be images or captions) given a query and supports both batched and streaming requests. A continuous integration and continuous development (CI/CD) pipeline (GitHub actions → Elastic Container Registry (ECR) → Elastic Container Service (ECS)) reconstructs the image whenever the main branch is updated, and a nightly retraining job in AWS Batch refreshes embeddings and the Approximate Nearest Neighbor (ANN) index. CloudWatch and Prometheus/Grafana (visual interface for monitoring and analyzing time-series data) dashboards provide latency / recall / drift monitoring. Estimated first-year spend is probably \$37/month at 1 requests/sec peak.

2. Deployment Options Considered

We looked at five different ways to serve our FAISS-powered search service:

1. **Lambda & API Gateway** seemed appealing at first due to zero-ops and essentially free when idle, but the tiny 512 MB memory limit (and lack of GPU) meant each cold-start would take several seconds, ruling it out for a sub 100 ms ANN use case.
2. **SageMaker Real-Time Endpoints** would give us managed autoscaling and built-in A/B testing, but the steep per-hour fees, proprietary lock-in, and generally bulky container startup times made it overkill for our MVP.
3. **Self-hosted Kubernetes (e.g. GKE)** offers the finest control given any image, any node type, but at the cost of a heavy operational burden: cluster provisioning,

node upgrades, logging/monitoring agents, and ongoing patching. I decided it was too complex for our scope.

4. **CS on Fargate** looks good: pay-per-second pricing, seamless IAM integration, and auto-scaling without managing servers. The cold-start latency is slightly higher than a dedicated cluster, but still within our SLO, so I chose this as our production target.
5. **On-prem GPU** could eliminate cloud spend and give full hardware control, but the upfront capital expenditure (CapEx), custom monitoring, and maintenance would be disproportionate for an amateur-scale prototype.

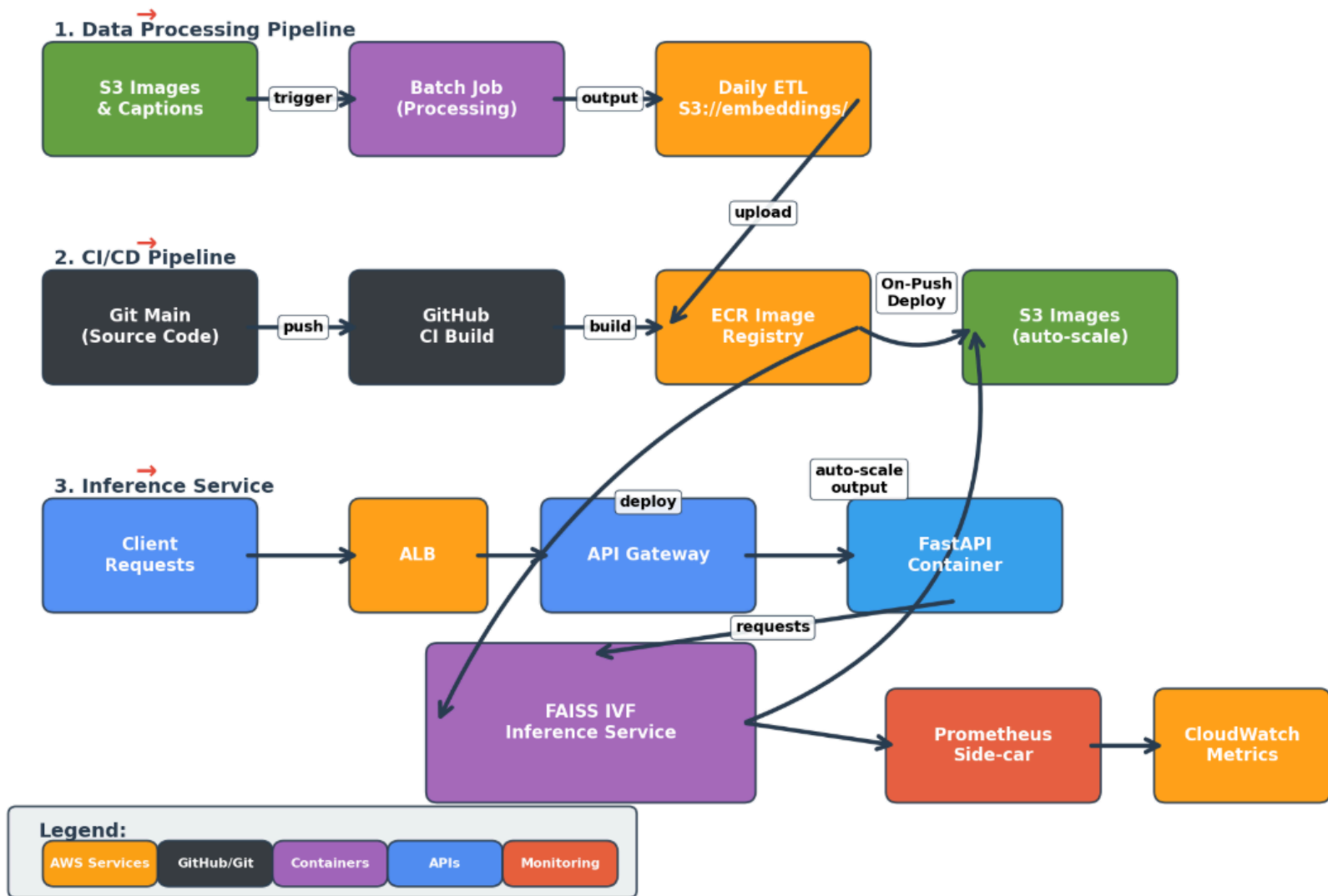
Option	Pros	Cons	Verdict
Lambda & API Gateway	Zero-ops, cheap idle	512 MB RAM cap, no GPU, over 5 seconds for Cold-start	Too slow for ANN
SageMaker Real-Time Endpoint	Managed scaling, built-in A/B	\$\$\$, vendor-lock, slow container builds	Overkill / unnecessary
GKE / Self-hosted Kubernetes (K8s)	Fine-grained control	Steep ops burden, cluster maintenance	Too MVP heavy.
ECS Fargate	Pay-per-second, no servers, IAM native	Slightly slower scale-out than K8s	Selected
On-premise GPU	No cloud bill, full control	CapEx, home-grown monitoring	Not suitable / unnecessary complexity for project scale and purpose

3. Chosen Architecture

This diagram, I made in Python, shown below, outlines how we'll glue everything together end-to-end: each night, a batch job pulls our raw images and captions out of S3, runs them through the embedding model, and writes fresh vectors back to an "embeddings" bucket. Meanwhile, our CI/CD pipeline watches for code changes, automatically building and pushing a new Docker image into ECR. On every deploy, Fargate spins up FastAPI containers behind an ALB → API Gateway that forward each incoming search request to our FAISS IVF service. We package Prometheus alongside to scrape latency and error metrics, then ship those into

CloudWatch for dashboarding and alerting. In summary, it's an automated flow from raw data to live, monitored retrieval service.

ML Pipeline Architecture



Step-by-step

1. Offline Pipeline

Every night (or on a schedule), a batch job pulls raw images & captions from S3, runs them through the CLIP/embedding model, and writes the resulting vectors

back to an S3 “embeddings” bucket.

2. CI/CD

- a. When we push to **main**, GitHub Actions builds a new Docker image containing our FastAPI + FAISS IVF server.
- b. The image is pushed to AWS ECR.
- c. A deployment hook then updates our service (ECS or Kubernetes) with the new image.

3. Online Serving

- a. Clients hit an Application Load Balancer (terminating TLS), which forwards requests to API Gateway.
- b. API Gateway invokes our FastAPI service, which uses FAISS IVF to retrieve the top-k nearest neighbors from the precomputed embeddings in S3.
- c. FastAPI returns the URLs of the matching source images in S3.

4. Observability & Scaling

- a. A Prometheus side-car alongside our FastAPI container scrapes query latency, QPS, and FAISS performance metrics.
- b. These are shipped to CloudWatch for dashboarding & alerting.
- c. The service auto-scales based on CPU, memory, and custom latency/SLO metrics.

Batch job (AWS Batch) nightly: encode new images/captions with ViT-B-32, rebuild `ivf_flat.index`, push to S3.

Serving FastAPI: loads the index on start, pins it in RAM (~2 GB). Query flow:

1. Tokenize input (OpenCLIP tokenizer), encode text/image to a 512-dimensional vector
2. `faiss.search(k) → ids` (IVF-Flat nlist = 1024, nprobe = 16)

3. Fetch the metadata from DynamoDB (id → URL/caption) and return JSON.

Scaling: Fargate tasks (CPU 1 vCPU, RAM 4 GB) auto-scale on p95 latency (performance metric that measures the response time for which 95% of requests are faster) > 150 ms or CPU > 70%

Monitoring: Prometheus side-car scrapes /metrics; Grafana dashboards alert on

1. P95 latency, error-rate, FAISS recall on canary checks
2. Embedding drift = $\cos\text{-sim}(\text{new batch}, \text{train PCA}) < 0.85$

Retraining / Redeploy: If drift > 10% for 3 days, GitHub Action triggers retrain notebook → new index → blue/green deploy.

4. Engineering Details & Pseudocode

```
# inference.py (FastAPI)
faiss_index = faiss.read_index('/data/ivf_flat.index')

@app.post("/search")
def search(query: Union[str, UploadFile], k:int=10):
    vec = encode(query)                # -> (512,)
    vec = vec / np.linalg.norm(vec)
    D, I = faiss_index.search(vec[None], k)
    results = [dynamodb_get(id_) for id_ in I[0]]
    return {"neighbors": results, "dist": D[0].tolist()}
```

CI/CD (.github/workflow/deploy.yml)

```
on: [push]
jobs:
  build:
    steps:
      - uses: actions/checkout@v3
      - run: docker build -t $IMAGE .
      - run: aws ecr push $IMAGE
      - run: aws ecs update-service --force-new-deployment ...
```

In the FastAPI service (`inference.py`), we first load a pre-built FAISS index from disk (`ivf_flat.index`) so it's resident in memory when the app starts. When a client POSTs to `/search`, we accept either a text string or an uploaded file, run it through our `encode(...)` function to get a 512-dimensional embedding, normalize that vector, and then call `faiss_index.search` to retrieve the top k nearest neighbors. FAISS returns both the distances (`D`) and the item IDs (`I`), so we then look up each ID in DynamoDB (via `dynamodb_get`) and return a JSON payload containing the neighbor records plus their distances.

On the CI/CD side, our GitHub Actions workflow (`deploy.yml`) is configured to run on every push to the repository. It checks out the code, builds a Docker image tagged with our ECR repository URI, and pushes that image up to Amazon ECR. Finally, it invokes the AWS CLI to force a new deployment of our ECS (or Kubernetes) service, ensuring the newly built container image is pulled and rolled out without any manual intervention.

5. Approximate Costs

Our end-to-end pipeline should run at a low cost. We'll probably pay about an estimate of \$1.76 per month to keep a single Fargate task (1 vCPU, 4 GB RAM) running 40 hours, plus roughly \$0.46/month to store 20 GB of embeddings and images in S3. The on-demand GPU batch job (`g4dn.xlarge`) used for training and periodic re-embedding runs about four hours a month, adding \$2.10. Finally, logging and metrics in CloudWatch come in around \$1.50/month. Altogether, we are looking at around \$5.82 per month to process, store, and serve our entire dataset with automated nightly updates.

Item	Units	Rate	Monthly
Fargate 1 x 1 vCPU/ 4 GB (40 h)	40 vCPU-hrs, 40 GB-hrs	\$0.040 / \$0.004	\$1.76
S3 storage embeddings & images (20 GB)	N/A	\$0.023/GB	\$0.46
Batch GPU job g4dn.xlarge (4 h/mo)	\$0.526 / hr	\$2.10	N/A
CloudWatch & data transfer	N/A	N/A	\$1.50
Total	N/A	N/A	Approximately $1.50 + 0.46 + 1.76 + 2.10 = \5.82

6. Post-Deployment Care

After launch, we'll have a 95th-percentile latency under 150 ms and $\geq 99.5\%$ availability. To have good performance, we'll monitor end-to-end latency, run a daily "recall canary" test to catch degradation, track HTTP 4xx/5xx errors, and (once we move to GPU inference) watch GPU memory and utilization. If recall drift or performance anomalies emerge, we'll trigger an urgent retraining cycle otherwise, we'll retrain weekly to keep the model fresh. New versions will be rolled out via ECS task-set blue/green deployments behind the ALB, allowing for easy rollback by flipping a Terraform `deploy_version` variable back to the last working ECR tag. Finally, we'll maintain a detailed runbook, on-call schedule, and cost dashboard (via AWS Cost Explorer) so we can diagnose incidents quickly and keep an eye on our expenditures.

- **Service SLO:** $p95 \leq 150$ ms, availability $\geq 99.5\%$.
- **Monitoring:** latency, recall canary, 4xx/5xx, GPU mem, drift.
- **Retraining cadence:** weekly or sooner if drift alert fires.
- **A/B rollout:** ECS task set with weighted ALB target groups.
- **Rollback:** terraform `deploy_version` variable $\rightarrow$ previous ECR tag.
- **Documentation:** run-book, on-call rotation, cost dashboard (AWS Cost Explorer).

7. Conclusion

Our end-to-end deployment design delivers an image-to-text retrieval service with minimal burden. Automated daily embedding updates and a fully managed CI/CD pipeline ensure that both data and code stay up to date, while monitoring, canary rollouts, and rollback procedures keep the service reliable and maintainable.